

NYU Fall Semester 2023

Lecture 21: File Systems II

Professor G. Sandoval

21

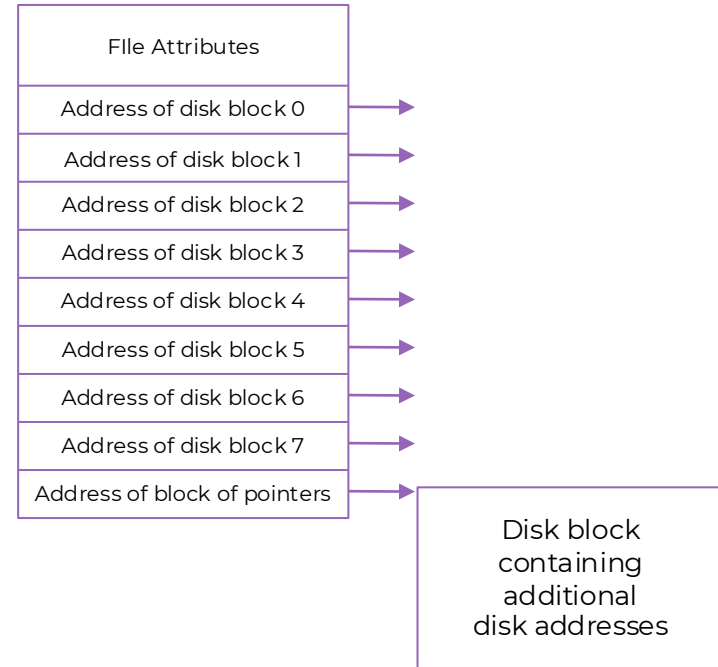


Some slides adapted by G. Sandoval for CS3224,
from slide by Brendan Dolan-Gavitt

I-Nodes

- A more compact way is to give each file a small data structure that lists its blocks, called an *i-node (index node)*
- Now we only need to store in memory an amount of data **proportional** to the number of open files
- Amount of storage needed to store the mapping **grows with the number of files**, *not* the total size of the disk

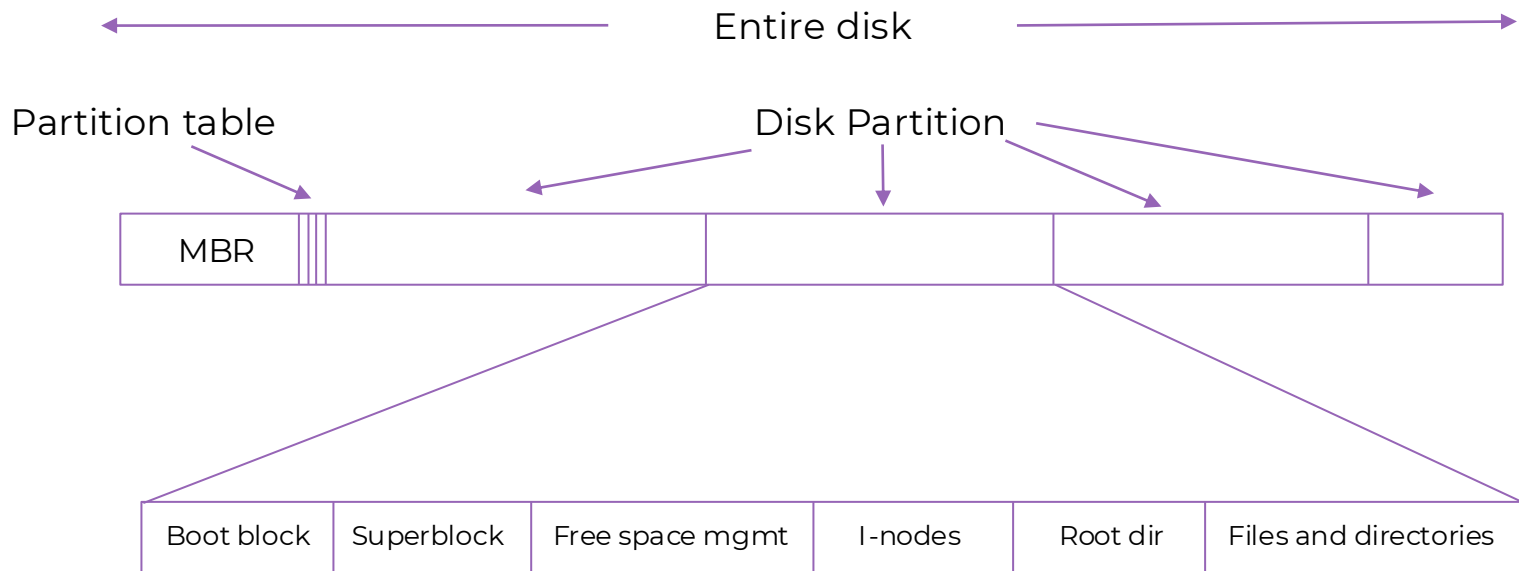
I-Nodes



Filesystem Layout on Disk

- A filesystem is defined by the layout of data on disk and the algorithms used to store and retrieve that data
- A typical disk layout consists of
 - A *master boot record* that contains code for booting the system
 - Multiple partitions, each with their own *boot block*
 - Within a partition, the actual filesystem data, usually beginning with a *superblock* that has key filesystem parameters

One Filesystem Layout



Example Superblock (xv6)

```
struct superblock {  
    uint size;                // Size of file system image  
    (blocks)  
    uint nblocks;            // Number of data blocks  
    uint ninodes;            // Number of inodes.  
    uint nlog;                // Number of log blocks  
    uint logstart;           // Block number of first log block  
    uint inodestart;          // Block number of first inode block  
    uint bmapstart;          // Block number of first free map  
    block  
};
```

Today

- File System Implementation (Cont)
 - Log Structured Systems
 - Journaling
- XV6 File System Layers
 - Buffer Cache Layer
 - Logging Layer
 - Inode Layer
 - Directory Layer
 - PathName Layer
 - File Descriptor

Write Performance Issues

- Most disks now have fast caches that make reads fast
- Writing is still slow – and worse because typical workload consists of many small writes
 - To write a file, you need to write to the directory entry, i-node, and finally file data
- Overhead dominates: the actual write may take only 50 microseconds, but requires:
 - 10ms to seek (move the read arm into the right position)
 - 4ms for disk platters to rotate into the right position

Log-Structured Filesystem

- To solve this, Rosenblum and Ousterhout (1992) created the *log-structured filesystem*
- Basic idea - structure the disk as **one big log**
- Periodically data **buffered** in memory collected into a single segment and **written** to the disk.

Log-Structured Filesystem

- Buffer writes in memory, then periodically flush them out in one contiguous segment at the end of the log
- At the start of the segment, place information about where to find i-nodes and file data within the current log segment
- Finally, maintain a map that says which log segment the i-nodes for files/directories can be found

Log-Structured Filesystem

- If we overwrite data in a file, or if we delete it, we will waste space (and eventually run out of space)
- To solve this, a *cleaner thread* constantly runs, removing unused entries from the back of the log and placing old but still-in-use entries at the front

Log-Structured Filesystem

- Structuring as a log also provides *crash recovery*
- If the OS crashes or we lose power in a traditional filesystem, we may leave things in an inconsistent state
 - For example: wrote file data, but didn't update the directory entry
- LFS solves this by keeping a *checkpoint*, which tracks what the most recent consistent filesystem state is
- After a crash, we can either just *revert* to the last *checkpoint*, or *revert and then replay* as many log entries as possible (while keeping the filesystem consistent)

Journaling Filesystems

- LFS is a major change to how filesystems work and hasn't been widely adopted
- However, one of its key ideas – using a log to provide recovery in the case of a crash – has been incorporated into modern filesystems
- These are called *journaling filesystems*
 - NTFS, HFS+, and ext3 all support journaling
 - ReiserFS was the first Linux FS to support journaling

Crashes and Inconsistency

- Crashes can lead to inconsistent filesystem states
- Consider operations to delete a file
 - Remove the file from the directory entry
 - Mark the i-node as free
 - Mark the file data blocks as free
- What happens if we do some of these and not the others because of a system crash?

Journaling

- Instead of just doing these three writes operations, first write a log entry to the *journal* saying what operations you're about to do
- Now if we crash, we can look at the journal and re-run the operations listed
- Once we're done, we can delete the log entry
- At worst, when the system crashes, we might end up repeating some operations

Idempotence

- Because the operations listed in the journal log entry could be carried out more than once, they must be *idempotent* (doing it twice is the same as doing it once)
- For example, marking a block as free in a bitmap is idempotent
- Adding a block to a list of free blocks is *not* idempotent
- But we can make it idempotent by adding a check to make sure it's not already in the list

In summary

- The key distinction is what gets written and when
- **Journaling filesystems** use a *small, dedicated log* as a **safety net** — they write metadata (and sometimes data) to a journal before applying changes to the main filesystem, then discard the journal entry once the real write completes. The journal is transient and circular.
- **Log-structured filesystems (LFS)** treat the **entire disk as a log** — *all* writes (data + metadata) are always appended sequentially to the end of a growing log. There is no "main filesystem" separate from the log; the log *is* the filesystem.

Side by side

Property	Journaling (e.g., ext4)	Log-Structured (e.g., F2FS, NILFS2)
Where writes go	Journal first, then in-place to disk	Always appended to end of log
Log purpose	Crash recovery only	Primary storage structure
Log size	Small, fixed circular buffer	Grows; old segments GC'd
Random writes	In-place (seek-heavy on HDD)	Sequential (great for SSDs/flash)
Read path	Direct from on-disk location	Must consult inode map / segment index
Space reclamation	Immediate (overwrite)	Requires garbage collection
Complexity	Lower	Higher (GC, segment cleaning)

Log Structured FS

Student writes `essay.docx`:

1. New data blocks appended to current log segment: [seg 47, offset 12]
2. New inode written to node segment: [seg 12, offset 8]
3. Node Address Table (NAT) updated: inode #4821 → seg 12, offset 8
4. Old inode block at [seg 9, offset 3] marked "stale"

No in-place writes ever. When segment 9 fills up with stale blocks, the **garbage collector** compacts live data and recycles the segment. This is why F2FS is the default filesystem on Android — flash storage loves sequential writes.

Journaling FS

Student writes essay.docx:

1. Journal: "About to write inode #4821 and data block #8903"
2. Write data to block #8903 (in place)
3. Write inode update to block #4821 (in place)
4. Journal: "Done – commit record written"
5. Journal entry recycled

Crash between steps 1-3? On recovery, ext4 **replays the journal** and either completes or rolls back the operation. The journal itself is typically 128MB on a 1TB drive.

Filesystem Roster

ext3/ext4	Journaling	Workhorse of Linux; 3 journal modes
XFS	Journaling	Metadata-only journal; great for large files
NTFS	Journaling	Windows default; journals metadata
HFS+/APFS	Journaling / CoW hybrid	macOS; APFS adds copy-on-write
NILFS2	Log-structured	Linux; continuous snapshotting
F2FS	Log-structured	Flash-optimized; Android default
LFS (BSD)	Log-structured	The original Rosenblum & Ousterhout 1992 paper

Today

- File System Implementation (Cont)
 - Log Structured Systems
 - Journaling
- **XV6 File System Layers**
 - Buffer Cache Layer
 - Logging Layer
 - Inode Layer
 - Directory Layer
 - PathName Layer
 - File Descriptor

xv6 FS Design

Requirements:

- On-disk data structures to represent tree of files & directories
- Crash recovery (never end up with inconsistent fs)
- Concurrent access by multiple files
- Provide caching, since disks are slow

xv6 FS

- The filesystem is one of the most complex parts of xv6
- Organized into seven layers:

We've seen this already



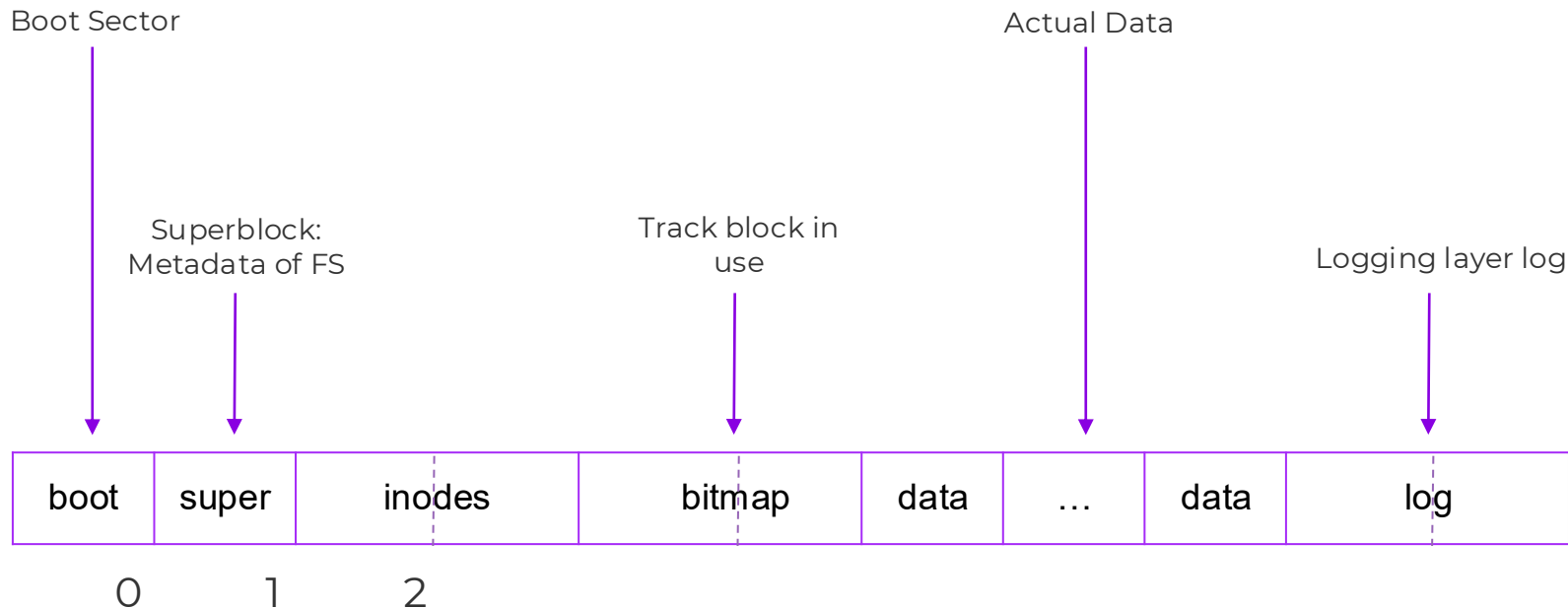
File descriptor
Pathname
Directory
Inode
Logging
Buffer cache
Disk

FS Layers

File descriptor
Pathname
Directory
Inode
Logging
Buffer cache
Disk

- **Disk layer** reads and writes blocks on the IDE drive
- **Buffer cache** layer caches blocks and synchronizes access to them
- **Logging layer** wraps multiple operations in a single atomic transaction, providing crash recovery
- **i-node layer** represents files as we saw last time
- **directory layer** contains a special type of i-node that gives a list of names and i-node pointers
- **pathname layer** resolves paths to i-nodes
- **file descriptor layer** provides an abstraction for accessing several kinds of object (pipes, files, devices) as files

Xv6 disk layout



Today

- File System Implementation (Cont)
 - Log Structured Systems
 - Journaling
- XV6 File System Layers
 - Buffer Cache Layer
 - Logging Layer
 - Inode Layer
 - Directory Layer
 - PathName Layer
 - File Descriptor

The Buffer Cache

File descriptor
Pathname
Directory
Inode
Logging
Buffer cache
Disk

- Two jobs:
 - **Synchronize** access to blocks so that two processes don't try to access the same data simultaneously
 - **Cache commonly** used blocks so that we don't have to read from the disk all the time
- Implemented in bio.c

High-Level Interface

- `bread()`:
 - Obtains a buffer containing a block that can be read or modified in memory.
- `bwrite()`:
 - Writes a modified buffer to disk.
- `brelse()`:
 - Called when done with a buffer to clear the B_BUSY flag

Buffer Cache Structure

```
struct buf {
    int flags;
    uint dev;
    uint blockno;
    struct buf *prev; // LRU cache list
    struct buf *next;
    struct buf *qnext; // disk queue
    uchar data[BSIZE];
};

#define B_BUSY 0x1 // buffer is locked by some process
#define B_VALID 0x2 // buffer has been read from disk
#define B_DIRTY 0x4 // buffer needs to be written to disk

struct bcache {
    struct spinlock lock;
    struct buf buf[NBUF];
    // Linked list of all buffers, through prev/next.
    // head.next is most recently used.
    struct buf head;
};
```

Linked list used for IDE queue

Buffer Read/Write

```
// Return a B_BUSY buf with the contents of the indicated block.
struct buf*
bread(uint dev, uint blockno)
{
    struct buf *b;
    b = bget(dev, blockno);
    if(!(b->flags & B_VALID)) {
        iderw(b);
    }
    return b;
}

// Write b's contents to disk.  Must be B_BUSY.
void
bwrite(struct buf *b)
{
    if((b->flags & B_BUSY) == 0)
        panic("bwrite");
    b->flags |= B_DIRTY;
    iderw(b);
}
```

Buffer Cache Synchronization

- Calling `bread()` attempts to get the block either from cache or by reading the disk
- It acquires a lock on the buffer cache, then tries to find a cached copy
- If the cached copy is found and not in use, it sets a flag (`B_BUSY`) on it, releases the lock, and returns
- If it is in use, it calls `sleep()` to wait until the block is free

Buffer Cache Synchronization (1/2)

```
// Look through buffer cache for block on device dev.  
// If not found, allocate a buffer.  
// In either case, return B_BUSY buffer.
```

```
static struct buf*  
bget(uint dev, uint blockno)  
{
```

```
    struct buf *b;
```

```
    acquire(&bcache.lock);
```

Access to cache protected by lock

```
loop:
```

```
// Is the block already cached?
```

```
for(b = bcache.head.next; b != &bcache.head; b = b->next){  
    if(b->dev == dev && b->blockno == blockno){  
        if(!(b->flags & B_BUSY)){  
            b->flags |= B_BUSY;  
            release(&bcache.lock);  
            return b;  
        }
```

```
        sleep(b, &bcache.lock);
```

```
        goto loop;  
    }
```

Sleep if block is found but busy

```
}
```

Buffer Cache Synchronization (2/2)

```
// Not cached; recycle some non-busy and clean buffer.
// "clean" because B_DIRTY and !B_BUSY means log.c
// hasn't yet committed the changes to the buffer.
for(b = bcache.head.prev; b != &bcache.head; b = b->prev){
    if((b->flags & B_BUSY) == 0 && (b->flags & B_DIRTY) ==
0){
        b->dev = dev;
        b->blockno = blockno;
        b->flags = B_BUSY;
        release(&bcache.lock);
        return b;
    }
}
panic("bget: no buffers");
}
```

Note: this is only safe because we already know there is no buf with this blockno and still have lock

Note: clears B_VALID and B_DIRTY

Buffer Caching

- To actually cache frequently used blocks, xv6 maintains an LRU cache
- Implemented as a doubly linked list (**prev** and **next** pointers in the **struct buf**)
- After releasing a block, we move it to the head of the list so it can be found quickly

LRU

```
// Look through buffer cache for block on device dev.  
// If not found, allocate a buffer.  
// In either case, return B_BUSY buffer.  
static struct buf*  
bget(uint dev, uint blockno)  
{  
    [...]  
    // Is the block already cached?  
    for(b = bcache.head.next; b != &bcache.head; b = b->next){  
        [...]  
    }  
    // Not cached; recycle some non-busy and clean buffer.  
    // "clean" because B_DIRTY and !B_BUSY means log.c  
    // hasn't yet committed the changes to the buffer.  
    for(b = bcache.head.prev; b != &bcache.head; b = b->prev){  
        [...]  
    }  
}
```

Search for cached block starts
at the head

Search for free block starts at the tail

Releasing a Buffer

```
// Release a B_BUSY buffer.
// Move to the head of the MRU list.
void
brelse(struct buf *b)
{
    if((b->flags & B_BUSY) == 0)
        panic("brelse");

    acquire(&bcache.lock);

    b->next->prev = b->prev;

    b->prev->next = b->next;
    b->next = bcache.head.next;
    b->prev = &bcache.head;
    bcache.head.next->prev = b;
    bcache.head.next = b;

    b->flags &= ~B_BUSY;
    wakeup(b);

    release(&bcache.lock);
}
```

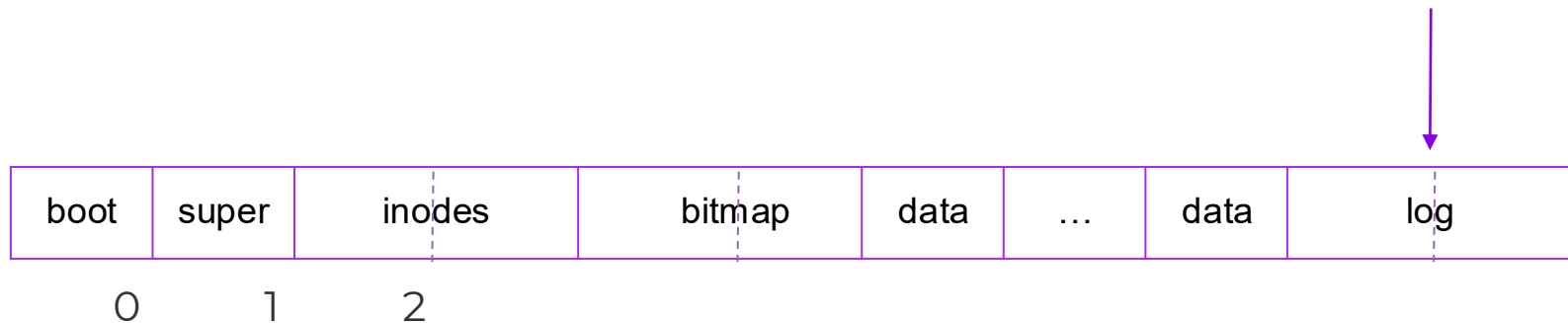
Clear B_BUSY flag

Wake up anyone
waiting on the
buffer

Today

- File System Implementation (Cont)
 - Log Structured Systems
 - Journaling
- XV6 File System Layers
 - Buffer Cache Layer
 - Logging Layer
 - Inode Layer
 - Directory Layer
 - PathName Layer
 - File Descriptor

Xv6 disk layout



Logging Layer

File descriptor
Pathname
Directory
Inode
Logging
Buffer cache
Disk

- Operations on the filesystem often require **multiple writes** to disk
- Crashes may leave things in an **inconsistent state**, e.g. during deletion:
 - May end up with directory entry pointing to a freed block (bad!)
 - May end up with a block that is not marked free but not referenced (harmless but wasteful)
- Logging layer allows higher layers to group multiple writes into a **single transaction** that will be committed **atomically**

Logging

- Instead of performing modifications to the disk directly, system calls in xv6 *write out a log* of pending operations to the disk
 - Each log entry consists of the data we intend to copy and the destination
- Once all modifications have been written to the log, write a special *commit* record
- Now, if there is a crash and the log is complete (has a commit record) we can replay it to recover

Log Layout

- Log is fixed size and at **fixed location** on the disk
- **Starts with a header**, which gives a count of the number of block writes pending as well as an array listing what the block numbers are
- Then, LOGSIZE blocks with the pending write data

Log Layout

Header
Count = 3
Block:
[212, 55, 376]

Block 212 Data

Block 55 Data

Block 376 Data

Committing

- Once a transaction is complete, xv6 writes the updated log header to disk
- When it has finished writing out all the logged blocks to disk, it sets the count in the log header to 0
- Now two possibilities when we crash:
- Partway through creating a transaction: count is zero, transaction is ignored
- Partway through writing: count is non-zero, so we redo the copy

Group Commit

- Multiple system calls can write to the log at once, allowing some concurrency
- So one log transaction may include data from multiple system calls
- xv6 makes sure not to actually commit until the last concurrent call finishes

Logging Code

- Interface exposed
 - `begin_op` – start a transaction (may sleep)
 - `write_log` – write a disk block (replaces `bwrite`)
 - `end_op` – ends a transaction
- Logging layer uses `bread/bwrite/brelse` (buffer cache layer)
- Logging code implemented in `log.c`

Using Logging

```
struct buf *bp;  
  
begin_op();  
[...]  
  
bp = bread(...);  
// modify bp->data  
  
log_write(bp);  
  
brelse(bp);  
[...]  
  
end_op();
```

Logging Structures

```
// Contents of the header block, used for both the on-disk header block
// and to keep track in memory of logged block# before commit.
struct logheader {
    int n;
    int block[LOGSIZE];
};

struct log {
    struct spinlock lock;
    int start;
    int size;
    int outstanding; // how many FS sys calls are executing.
    int committing; // in commit(), please wait.
    int dev;
    struct logheader lh;
};

struct log log;
```

Init Log

```
void
initlog(int dev)
{
    if (sizeof(struct logheader) >= BSIZE)
        panic("initlog: too big logheader");

    struct superblock sb;
    initlock(&log.lock, "log");
    readsb(dev, &sb);
    log.start = sb.logstart;
    log.size = sb.nlog;
    log.dev = dev;
    recover_from_log();
}
```

Beginning a Transaction

```
// called at the start of each FS system call.
// #define MAXOPBLOCKS 10 // max # of blocks any FS op writes
// #define LOGSIZE      (MAXOPBLOCKS*3) // max data blocks in on-disk log
// #define NBUF          (MAXOPBLOCKS*3) // size of disk block cache
void
begin_op(void)
{
    acquire(&log.lock);
    while(1){
        if(log.committing){
            sleep(&log, &log.lock);
        } else if(log.lh.n + (log.outstanding+1) * MAXOPBLOCKS >
LOGSIZE){
            // this op might exhaust log space; wait for commit.
            sleep(&log, &log.lock);
        } else {
            log.outstanding += 1;
            release(&log.lock);
            break;
        }
    }
}
```

Wait until system is not committing

If this txn will get us over log space => sleep

Writing to the Log

```
// Caller has modified b->data and is done with the buffer.
// Record the block number and pin in the cache with B_DIRTY.
// commit()/write_log() will do the disk write.
//
log_write(struct buf *b)
{
    int i;

    if (log.lh.n >= LOGSIZE || log.lh.n >= log.size - 1)
        panic("too big a transaction");
    if (log.outstanding < 1)
        panic("log_write outside of trans");

    acquire(&log.lock);
    for (i = 0; i < log.lh.n; i++) {
        if (log.lh.block[i] == b->blockno)    // log absorption
            break;
    }
    log.lh.block[i] = b->blockno;
    if (i == log.lh.n)
        log.lh.n++;
    b->flags |= B_DIRTY; // prevent eviction
    release(&log.lock);
}
```

Records block num

Mark buffer as dirty

Ending a Transaction

```
// called at the end of each FS system call.
// commits if this was the last outstanding operation.
void
end_op(void)
{
    int do_commit = 0;

    acquire(&log.lock);
    log.outstanding -= 1;
    if(log.committing)
        panic("log.committing");
    if(log.outstanding == 0){
        do_commit = 1;
        log.committing = 1;
    } else {
        // begin_op() may be waiting for log space.
        wakeup(&log);
    }
    release(&log.lock);

    if(do_commit){
        // call commit w/o holding locks, since not allowed
        // to sleep with locks.
        commit();
        acquire(&log.lock);
        log.committing = 0;
        wakeup(&log);
        release(&log.lock);
    }
}
```

Commit

```
static void
commit()
{
    if (log.lh.n > 0) {
        write_log();           // Write modified blocks from cache to
log
        write_head();          // Write header to disk -- the real commit
        install_trans();       // copy from log area to diskhome locations
        log.lh.n = 0;
        write_head();          // Erase the transaction from the log
    }
}
```

write_log

```
// Copy modified blocks from cache to log.
static void
write_log(void)
{
    int tail;

    for (tail = 0; tail < log.lh.n; tail++) {
        struct buf *to = bread(log.dev, log.start+tail+1); // log block
        struct buf *from = bread(log.dev, log.lh.block[tail]); // cache block
        memmove(to->data, from->data, BSIZE);
        bwrite(to); // write the log
        brelse(from);
        brelse(to);
    }
}
```

write_head

```
// Write in-memory log header to disk.
// This is the true point at which the
// current transaction commits.
static void
write_head(void)
{
    struct buf *buf = bread(log.dev, log.start);
    struct logheader *hb = (struct logheader *) (buf->data);
    int i;
    hb->n = log.lh.n;
    for (i = 0; i < log.lh.n; i++) {
        hb->block[i] = log.lh.block[i];
    }
    bwrite(buf);
    brelse(buf);
}
```

install_trans

```
// Copy committed blocks from log to their home location
// Until you call install_trans nothing changes on the OS
static void
install_trans(void)
{
    int tail;

    for (tail = 0; tail < log.lh.n; tail++) {
        struct buf *lbuf = bread(log.dev, log.start+tail+1); // read log block
        struct buf *dbuf = bread(log.dev, log.lh.block[tail]); // read dst
        memmove(dbuf->data, lbuf->data, BSIZE); // copy block to dst
        bwrite(dbuf); // write dst to disk
        brelse(lbuf);
        brelse(dbuf);
    }
}
```

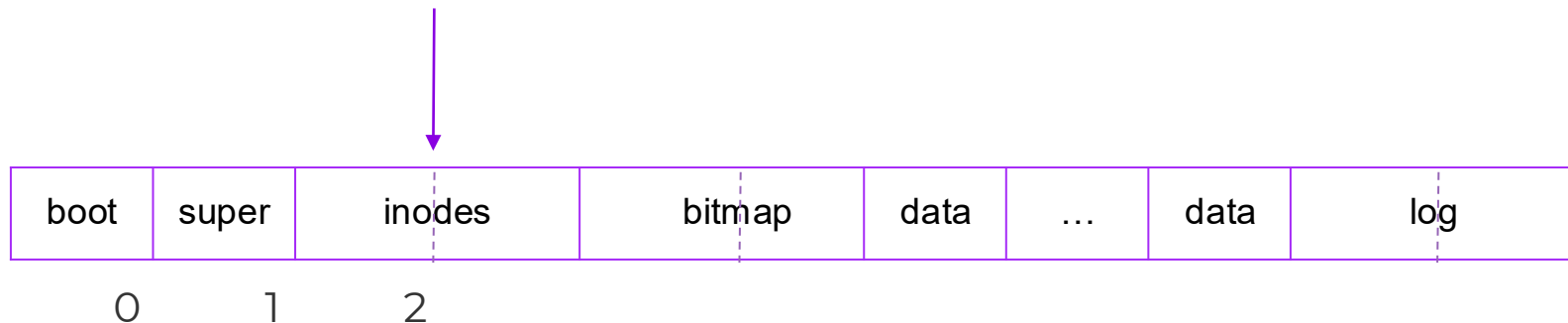
Crash Recovery

```
// Called from init_log which is called during boot
// Before the first process runs
static void
recover_from_log(void)
{
    read_head();
    install_trans();           // if committed, copy from log to
disk
    log.lh.n = 0;
    write_head();             // clear the log
}
```

Today

- File System Implementation (Cont)
 - Log Structured Systems
 - Journaling
- XV6 File System Layers
 - Buffer Cache Layer
 - Logging Layer
 - Inode Layer
 - Directory Layer
 - PathName Layer
 - File Descriptor

Xv6 disk layout



i-node Layer

File descriptor
Pathname
Directory
Inode
Logging
Buffer cache
Disk

- i-nodes store pointers to file blocks
- Every i-node is the same size and they are stored in a single area on disk, so it's easy to look up an i-node by number:
$$\text{inode_start} + \text{inode_num} / \text{inodes_per_block}$$
- i-nodes in xv6 have both an on-disk (`struct dinode`) and in-memory representation (`struct inode`)
- xv6 maintains a cache of in-memory i-nodes in order to help synchronize access to i-nodes by multiple processes

i-node Interface

- `iget` – gets an i-node from the cache, does not guarantee that access is exclusive (many processes can access at once)
- `iput` – releases the i-node, decrementing its reference count
- `ilock` – actually reads in the i-node data from disk (if not already present) and gets exclusive access
- `iunlock` – releases the lock on the i-node

i-node Structures

```
// in-memory copy of an inode
struct inode {
    uint dev;           // Device number
    uint inum;          // Inode number
    int ref;            // Reference count
    int flags;          // I_BUSY, I_VALID

    short type;         // copy of disk inode
    short major;
    short minor;
    short nlink;
    uint size;
    uint addrs[NDIRECT+1];
};

// On-disk inode structure
struct dinode {
    short type;         // File type (file, dir, or device. 0 means it's free)
    short major;        // Major device number (T_DEV only)
    short minor;        // Minor device number (T_DEV only)
    short nlink;        // Number of links to inode in file system
    uint size;          // Size of file (bytes)
    uint addrs[NDIRECT+1]; // Data block addresses
};
```

i-node Creation

- To create an i-node, we use `ialloc`
- Scans the on-disk i-nodes looking for one that's free
- **Remember:** The fact that the buffer layer guarantees exclusive access to a block means **we don't have to worry about another process claiming the same free i-node**

i-node Creation Code

```
// Allocate a new inode with the given type on device dev.
// A free inode has a type of zero.
struct inode*
ialloc(uint dev, short type)
{
    int inum;
    struct buf *bp;
    struct dinode *dip;

    for(inum = 1; inum < sb.ninodes; inum++){
        bp = bread(dev, IBLOCK(inum, sb));
        dip = (struct dinode*)bp->data + inum % IPB;
        if(dip->type == 0){ // a free inode
            memset(dip, 0, sizeof(*dip));
            dip->type = type; // set it to our type: file, dir/dev
            log_write(bp); // mark it allocated on the disk
            brelse(bp);
            return iget(dev, inum);
        }
        brelse(bp);
    }
    panic("ialloc: no inodes");
}
```

Example:File Creation (1/2)

```
int
sys_open(void)
{
    char *path;
    int fd, omode;
    struct file *f;
    struct inode *ip;

    if(argstr(0, &path) < 0 || argint(1, &omode) < 0)
        return -1;

    begin_op();

    if(omode & O_CREATE){
        ip = create(path, T_FILE, 0, 0);
        if(ip == 0){
            end_op();
            return -1;
        }
    }

    [...]
}
```

File Creation (2/2)

```
static struct inode*
create(char *path, short type, short major, short minor)
{
    uint off;
    struct inode *ip, *dp;
    char name[DIRSIZ];

[ ... Look up parent directory ... ]

    if((ip = ialloc(dp->dev, type)) == 0)
        panic("create: ialloc");

    ilock(ip);
    ip->major = major;
    ip->minor = minor;
    ip->nlink = 1;
    iupdate(ip);

[...]
```

iget

```
// Find the inode with number inum on device dev
// and return the in-memory copy. Does not lock
// the inode and does not read it from disk.
static struct inode*
iget(uint dev, uint inum)
{
    struct inode *ip, *empty;

    acquire(&icache.lock);

    // Is the inode already cached?
    empty = 0;
    for(ip = &icache.inode[0]; ip < &icache.inode[NINODE]; ip++){
        if(ip->ref > 0 && ip->dev == dev && ip->inum == inum){
            ip->ref++;
            release(&icache.lock);
            return ip;
        }
        if(empty == 0 && ip->ref == 0)    // Remember empty slot.
            empty = ip;
    }
    [ ... code to initialize the inode ...]
    release(&icache.lock);

    return ip;
}
```

iput

```
// Drops the ref count to an in-memory inode.
// If that was the last reference, the inode cache entry can
// be recycled.
// If that was the last reference and the inode has no links
// to it, free the inode (and its content) on disk.
// All calls to iput() must be inside a transaction in
// case it has to free the inode.
void
iput(struct inode *ip)
{
    acquire(&icache.lock);
    if(ip->ref == 1 && (ip->flags & I_INVALID) && ip->nlink == 0){
        // inode has no links and no other references: truncate and free.
        if(ip->flags & I_BUSY)
            panic("iput busy");
        ip->flags |= I_BUSY;
        release(&icache.lock);
        itrunc(ip);
        ip->type = 0;
        iupdate(ip);
        acquire(&icache.lock);
        ip->flags = 0;
        wakeup(ip);
    }
    ip->ref--;
    release(&icache.lock);
}
```

ilock

Note: copy inode from disk into memory

```
// Lock the given inode.
// Reads the inode from disk if necessary.
void
ilock(struct inode *ip)
{
    struct buf *bp;
    struct dinode *dip;

    if(ip == 0 || ip->ref < 1)
        panic("ilock");

    acquire(&icache.lock);
    while(ip->flags & I_BUSY)
        sleep(ip, &icache.lock);
    ip->flags |= I_BUSY;
    release(&icache.lock);

    if(!(ip->flags & I_VALID)){
        bp = bread(ip->dev, IBLOCK(ip->inum, sb));
        dip = (struct dinode*)bp->data + ip->inum%IPB;
        ip->type = dip->type;
        ip->major = dip->major;
        ip->minor = dip->minor;
        ip->nlink = dip->nlink;
        ip->size = dip->size;
        memmove(ip->addrs, dip->addrs, sizeof(ip->addrs));
        brelse(bp);
        ip->flags |= I_VALID;
        if(ip->type == 0)
            panic("ilock: no type");
    }
}
```

iupdate

Note: write a modified inode back to disk

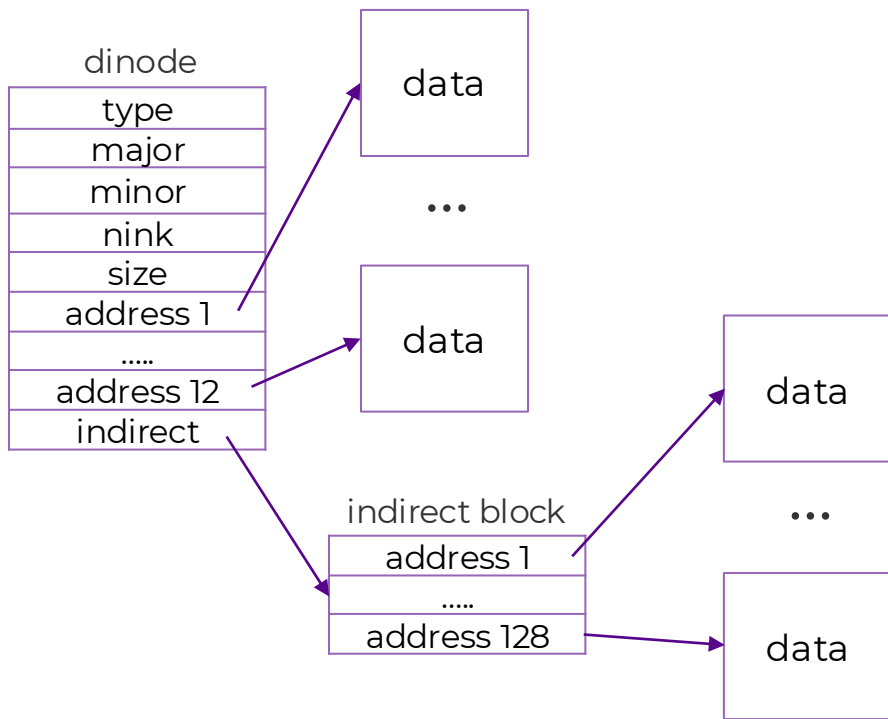
```
// Copy a modified in-memory inode to disk.
void
iupdate(struct inode *ip)
{
    struct buf *bp;
    struct dinode *dip;

    bp = bread(ip->dev, IBLOCK(ip->inum, sb));
    dip = (struct dinode*)bp->data + ip->inum%IPB;
    dip->type = ip->type;
    dip->major = ip->major;
    dip->minor = ip->minor;
    dip->nlink = ip->nlink;
    dip->size = ip->size;
    dip->created = ip->created;
    memmove(dip->addrs, ip->addrs, sizeof(ip->addrs));
    log_write(bp);
    brelse(bp);
}
```

iunlock

```
// Unlock the given inode.  
void  
iunlock(struct inode *ip)  
{  
    if(ip == 0 || !(ip->flags & I_BUSY) || ip->ref < 1)  
        panic("iunlock");  
  
    acquire(&icache.lock);  
    ip->flags &= ~I_BUSY;  
    wakeup(ip);  
    release(&icache.lock);  
}
```

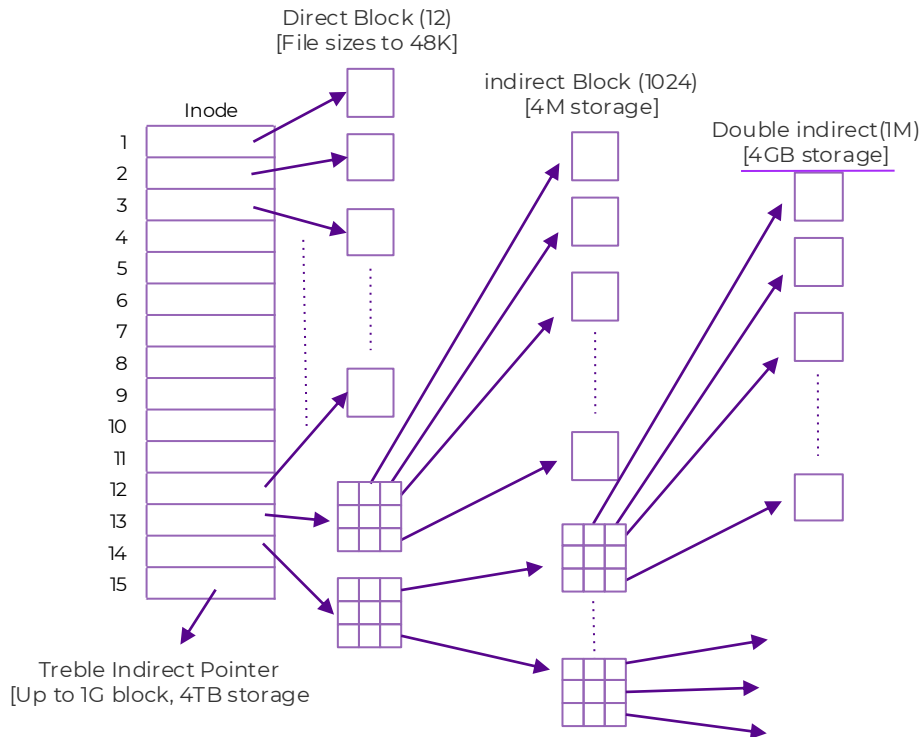
i-node Content



i-node Content

- Note that there's only *one* level of indirection here
- So there is a **maximum file size**:
(number of direct blocks +
number of indirect blocks) x
blocksize
= (12 + 128) x 512 bytes = 70 KB

ext3 i-nodes



Today

- File System Implementation (Cont)
 - Log Structured Systems
 - Journaling
- XV6 File System Layers
 - Buffer Cache Layer
 - Logging Layer
 - Inode Layer
 - Directory Layer
 - PathName Layer
 - File Descriptor

xv6 Directories

File descriptor
Pathname
Directory
Inode
Logging
Buffer cache
Disk

```
// Directory is a file containing a  
// sequence of dirent structures.
```

```
#define DIRSIZ 14
```

```
struct dirent {  
    ushort inum;  
    char name[DIRSIZ];  
};
```

- File & directory names are limited to 14 characters !
- Directories are just another kind of file, whose content is a list of `struct dirents`

Directory Lookup

```
// Look for a directory entry in a directory.
// If found, set *poff to byte offset of entry.
struct inode*
dirlookup(struct inode *dp, char *name, uint *poff)
{
    uint off, inum;
    struct dirent de;

    if(dp->type != T_DIR)
        panic("dirlookup not DIR");

    for(off = 0; off < dp->size; off += sizeof(de)){
        if(readi(dp, (char*)&de, off, sizeof(de)) != sizeof(de))
            panic("dirlink read");
        if(de.inum == 0)
            continue;
        if(namecmp(name, de.name) == 0){
            // entry matches path element
            if(poff)
                *poff = off;
            inum = de.inum;
            return iget(dp->dev, inum);
        }
    }
    return 0;
}
```

All we need to get a file is it's inode

Avoiding Deadlock

- The reason `iget` and `ilock` are separate functions is to help avoid deadlock
- In `dirlookup`, what happens if the thing we're looking up is "." ?
- The caller has locked the directory inode
- `Dirlookup` doesn't want it to be evicted from the cache
- But "." refers to the current directory, so attempting to lock it will cause a deadlock
- Solution: separate out acquisition and locking

Real World Filesystems

- Buffer caches can use more sophisticated eviction policies for particular workloads (priorities?)
- Logging can be made more efficient:
 - Don't log entire blocks (only a few bytes may have changed)
 - Allow filesystem calls to happen at the same time as a commit is taking place
- Use better data structures (e.g., balanced tree) for directories to avoid linear scan

Today

- File System Implementation (Cont)
 - Log Structured Systems
 - Journaling
- XV6 File System Layers
 - Buffer Cache Layer
 - Logging Layer
 - Inode Layer
 - Directory Layer
 - **PathName Layer**
 - File Descriptor

Path names

- Path Name lookup involves a succession of calls to `dirlookup`, one for each path component.
 - Think of a call like: `ls home\foo.txt`
- The function `namei` evaluates the path and returns the corresponding `inode`.
- `Namei` calls `namex` to do all the work of parsing the pathname and call .

Namei()

```
5539 struct inode*
5540 namei (char *path)
5541 }
5542     char name [DIRSIZ];
5543     return namex (path, 0, name);
5544 }
5545
```

Namex()

```
5500 // Look up and return the inode for a path name.
5501 // If parent != 0, return the inode for the parent and copy the final
5502 // path element into name, which must have room for DIRSIZ bytes.
5503 // Must be called inside a transaction since it calls iput ().
5504 static struct inode*
5505 namex (char *path, int nameparent, char *name)
5506 {
5507     struct inode *ip, *next;
5508
5509     if (*path == '/')
5510         ip = iget (ROOTDEV, ROOTINO);
5511     else
5512         ip = idup (proc->cwd);
5513
5514     while ((path = skipelem (path, name)) != 0) {
5515         ilock (ip);
5516         if (ip->type == T_DIR) {
5517             iunlockput (ip);
5518             return 0;
5519         }
5520         if (nameparent && *path == '\0'){
5521             // Stop one level early.
5522             iunlock (ip);
5523             return ip;
5524         }
5525         if ((next = dirlookup (ip, name, 0)) == 0) { ==
5526             iunlockput (ip);
5527             return 0;
5528         }
5529         iunlockput (ip);
5530         ip = next;
5531     }
5532     if (nameparent) {
5533         iput (ip);
5534         return 0;
5535     }
5536     return ip;
5537 }
```

Today

- File System Implementation (Cont)
 - Log Structured Systems
 - Journaling
- XV6 File System Layers
 - Buffer Cache Layer
 - Logging Layer
 - Inode Layer
 - Directory Layer
 - PathName Layer
 - File Descriptor

- XV6 gives each process it's own table of open files or file descriptors.
- Each open file is represented by a **struct file** which is a wrapper around an inode or a pipe.

```
4000 struct file {  
4001     enum { FD_NONE, FD_PIPE, FD_INODE } type;  
4002     int ref; // reference count  
4003     char readable;  
4004     char writable;  
4005     struct pipe *pipe;  
4006     struct inode *ip;  
4007     uint off;  
4008 };
```

- All the open files in the system are kept in a global file table the **ftable**.
- The file table has functions to
 - Allocate a file (**filealloc**)
 - Create a duplicate reference (**filedup**)
 - Release a reference (**fileclose**)
 - Read data (**fileread**)
 - Write data (**filewrite**)

```
5702 filestat(struct file *f, struct stat *st)
5703 {
5704     if (f->type FD_INODE) {
5705         ilock (f->ip);
5706         stati (f->ip, st);
5707         iunlock (f->ip);
5708         return 0;
5709     }
5710     return -1;
5711 }
```

```
5235 // Copy stat information from inode.
5236 void
5237 stati(struct inode *ip, struct stat *st)
5238 {
5239     st->dev = ip->dev;
5240     st->ino = ip->inum;
5241     st->type ip->type; =
5242     st->nlink ip->nlink; =
5243     st->size ip->size; =
5244 }
```